Name: János File

Neptun code: COXBBL

Secure Client-Server Chat using Diffie-Hellman and AES

1. Introduction

The objective of this project was to implement a secure, networked communication system between two parties using modern cryptographic standards. The assignment required combining the Diffie-Hellman key exchange algorithm with a symmetric encryption method (AES) over a Server-Client architecture.

To achieve this, the project was developed in Python 3 on a Linux environment (CachyOS). The implementation relies on Python's native `socket` library for network communication and `tkinter` for the graphical user interface. All cryptographic operations were handled using the standard Python `cryptography` library. The resulting application allows two users to exchange messages securely, ensuring confidentiality and integrity over an insecure local network connection.

2. Theoretical Background

Securing a chat application requires solving two distinct cryptographic problems: agreeing on a secret password over an open channel, and using that password to scramble the actual messages.

Diffie-Hellman Key Exchange (Asymmetric) Diffie-Hellman is a mathematical method that allows two parties who have never met to establish a shared secret over a public, monitored network. In this implementation, the server generates the mathematical parameters and a private/public key pair. It sends the parameters and public key to the client. The client uses the parameters to generate its own key pair and sends its public key back. Both sides then combine their private key with the other's public key. Mathematically, this results in both the server and the client independently calculating the exact same "Shared Secret."

AES-GCM Encryption (Symmetric) Once the shared secret is established, it is used to encrypt the chat. Advanced Encryption Standard (AES) is a symmetric cipher, meaning the same key is used to lock and unlock the data. This project uses AES in Galois/Counter Mode (GCM). GCM is highly secure because it not only encrypts the data to provide confidentiality, but it also authenticates the message. If any data is tampered with while traveling across the network, the decryption will fail, alerting the system.

3. Networking and Application Architecture

To ensure the application is versatile and testable across different environments, I implemented a dynamic input system using `tkinter.simpledialog`.

- **Port Selection:** Instead of a hardcoded port, the user can specify any available port (e.g., 65432). This prevents "Address already in use" errors by allowing the user to simply switch ports if a socket is stuck in a `TIME_WAIT` state.
- **IP Binding:** The server binds to `0.0.0.0`, allowing it to listen on all local network interfaces. The client prompts for a destination IP, enabling the application to work on `localhost` (127.0.0.1) for single-machine testing or a local network IP (e.g., 192.168.x.x) for multi-machine testing.

A significant architectural challenge in this project was combining network listening with a Graphical User Interface (GUI). By default, listening for network data blocks the execution of the program, which causes GUI windows to freeze. To resolve this, the application utilizes Python's `threading` library. The network connection and the message-receiving loop are pushed to a background daemon thread. This allows the `tkinter` interface to remain fully responsive for the user while the application silently listens for incoming encrypted packets in the background.

4. Cryptographic Implementation Details

4.1 The Key Exchange

The key exchange is initiated the moment the client connects. The server uses the `cryptography` library to generate 512-bit parameters. While 2048-bit is standard for production environments, 512-bit was chosen for this assignment to ensure rapid execution during testing.

Python
# Generating parameters and keys (Server side)
parameters = dh.generate_parameters(generator=2, key_size=512)
private_key = parameters.generate_private_key()
public_key = private_key.public_key()

4.2 Key Derivation

The Diffie-Hellman shared secret is a large integer, which cannot be used directly as an AES key. The application uses a secure hash function (`SHA-256`) to derive a uniform, 32-byte (256-bit) key suitable for AES-256.

Python

```
# Calculating shared secret and deriving AES key
shared_secret = private_key.exchange(peer_pub_key)
aes_key = hashlib.sha256(shared_secret).digest()
session["aes"] = AESGCM(aes_key)
```

4.3 Message Encryption

When a user sends a message, the application generates a 12-byte random "nonce" (number used once). This ensures that even if the exact same message is sent twice, the resulting ciphertext will look completely different, preventing replay attacks.

Python
```
# Encrypting outgoing messages
nonce = os.urandom(12)
ciphertext = session["aes"].encrypt(nonce, text.encode(), None)
session["conn"].sendall(nonce + ciphertext)
```

4.4 Message Decryption

Upon receiving a packet, the application splits the first 12 bytes to retrieve the nonce, and uses the AES-GCM object to decrypt and verify the remaining ciphertext.

Python
```
# Decrypting incoming messages
nonce = data[:12]
ciphertext = data[12:]
decrypted = session["aes"].decrypt(nonce, ciphertext, None)
```

5. Testing and Results

To verify the implementation, the server and client scripts were executed concurrently. The following sequence demonstrates the successful establishment of a secure session and encrypted data transfer.

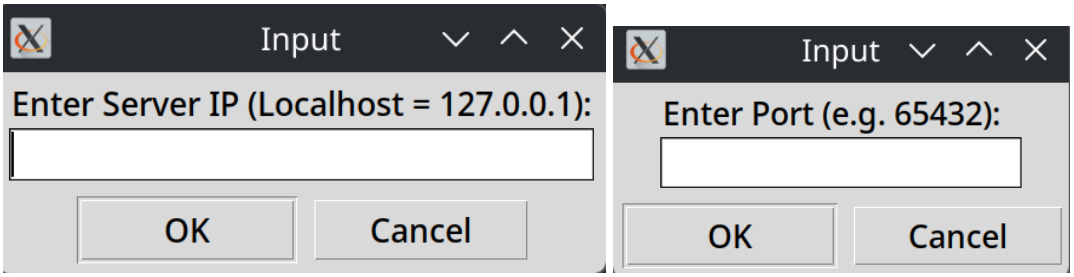

Figure 1: Dynamic network configuration prompts allowing for cross-machine interoperability.

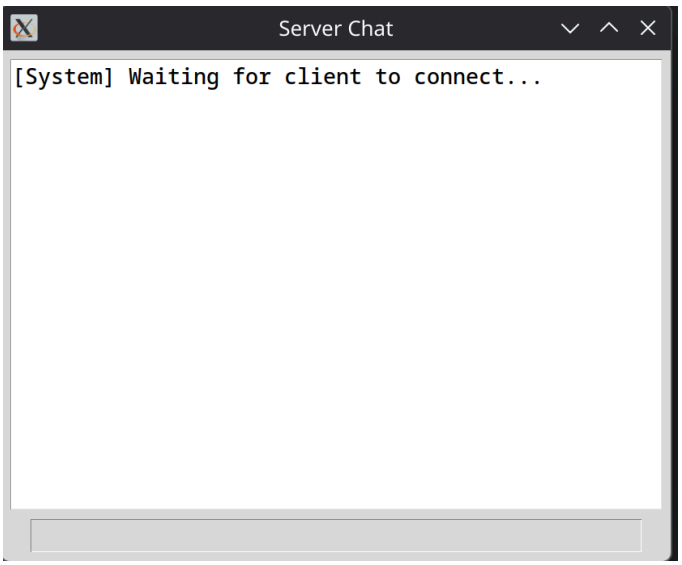

Figure 2: The server initializes the GUI and listens on port 65432, waiting for a client connection.

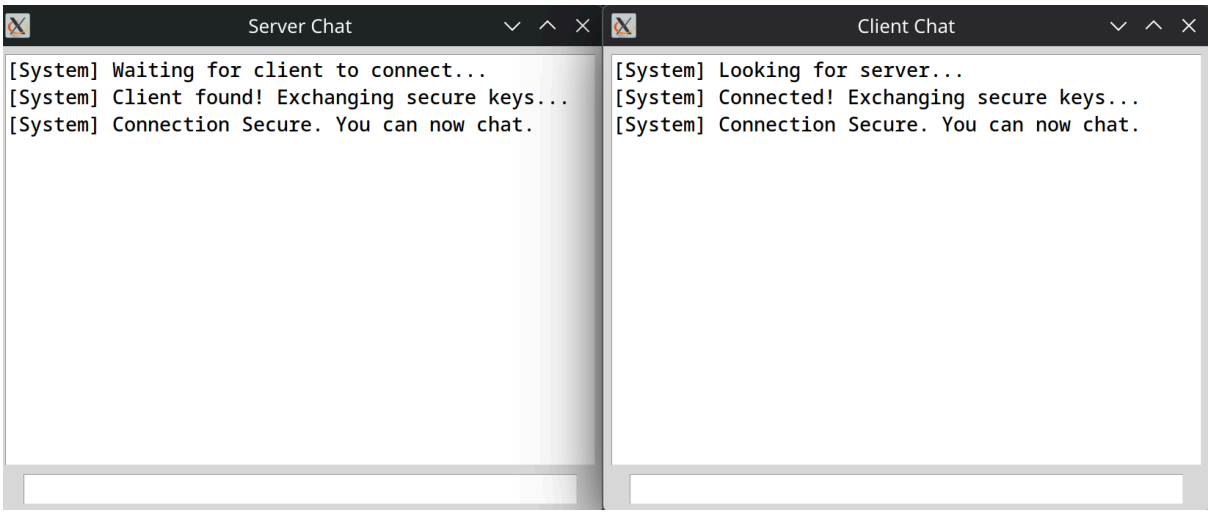

Figure 3: The client connects. Both parties successfully exchange Diffie-Hellman public keys, derive the AES key, and unlock the chat interface for communication.

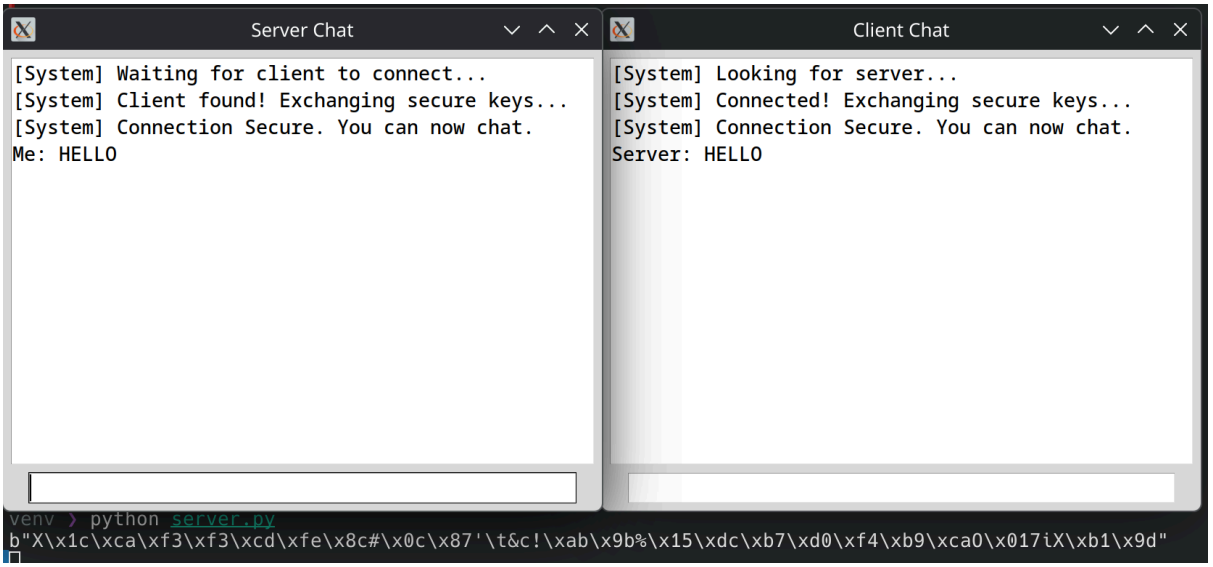

Figure 4: Two-way communication established. Messages are encrypted via AES-256-GCM before transmission and successfully decrypted upon arrival.

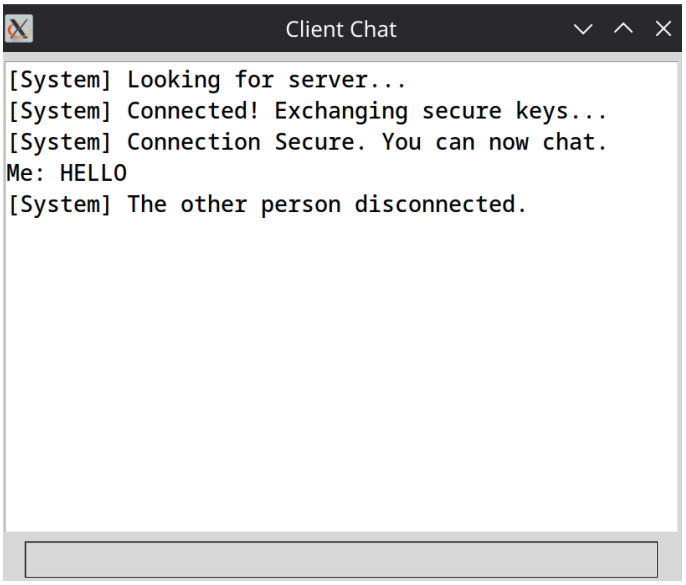

Figure 5: Graceful disconnection handling. When the server is closed, the client instantly detects the severed socket and disables the user input.

6. Conclusion

The implementation successfully meets all requirements of the assignment. The Diffie-Hellman algorithm successfully established a secure shared secret over an open TCP socket, and AES-GCM provided robust symmetric encryption for the messaging layer. Wrapping the logic in a multithreaded graphical interface demonstrated a complete, practical application of cryptographic concepts rather than a simple terminal script.